Arquitetura da Plataforma de Estudos

> **Status:** rascunho v0.2 · **Escopo:** estágio inicial (MVP até templates compartilháveis)
> Este documento define **como** o sistema é organizado, **quais regras** todo código deve seguir e **quais limites** a customização tem. Mudanças nele exigem um ADR (ver §15).
> **Como ler:** para uma visão rápida, leia §1 a §3, §8 e §17. As demais seções são referência para quando a implementação chegar naquela parte.

As palavras **DEVE**, **NÃO DEVE**, **DEVERIA** e **PODE** seguem o sentido da RFC 2119: *DEVE* é obrigatório e verificado em CI sempre que possível; *DEVERIA* é o padrão, com exceções justificadas em ADR.

Sumário

1. Propósito e visão
2. Princípios
3. Visão geral da arquitetura
4. Módulos e fronteiras
5. Núcleo de domínio
6. Metadados e templates
7. Registros (pontos de extensão)
8. Camadas de customização e limites
9. Motor de regras
10. Dados e persistência
11. Frontend, offline e sincronização
12. Segurança e privacidade
13. Extensibilidade
14. Ferramentas de desenvolvimento
15. Processo de mudança
16. Verificação automática (fitness functions)
17. Implementação por fases
18. Fora de escopo
19. Glossário

---

1. Propósito e visão

A plataforma ajuda pessoas a organizar a vida de estudos: compromissos de aula, revisões, prática de questões, anotações e metas. O diferencial é se adaptar à área de cada pessoa por meio de **templates** — um estudante de medicina marca plantões, um concurseiro marca simulados e acompanha o edital, um estudante de biologia marca saídas de campo.

A origem do projeto é o cronograma de estudos para a residência SES-DF (HTML único com `localStorage`). Nele, "plantão" e "revisão" estão fixos no código. **A tese desta arquitetura é que esses conceitos viram dados**, definidos por templates, sobre um núcleo genérico.

---

2. Princípios

P1 — Núcleo pequeno, metadados ricos. O núcleo conhece apenas conceitos que servem a qualquer estudante. Tudo que é específico de uma área vive em templates.

P2 — Customização por combinação, nunca por programação. Usuários combinam peças de catálogos fechados (tipos de campo, geradores, gatilhos, ações, widgets). Usuários não escrevem código.

P3 — Aberto para extensão, fechado para modificação. Adicionar um tipo de campo, gerador ou ação é criar uma implementação e registrá-la. Nenhum outro arquivo do sistema deveria precisar mudar.

P4 — Fronteiras explícitas e verificadas. Módulos só se comunicam por interface pública ou evento de domínio. Violações quebram o build.

P5 — Os dados do usuário são dele. Nenhuma atualização de template sobrescreve customizações. Nada é apagado sem ação explícita. Tudo é exportável.

P6 — Simplicidade operacional. Um deploy, um banco, nenhuma infraestrutura extra até que uma métrica real justifique.

---

3. Visão geral da arquitetura

Estilo: monolito modular, com arquitetura hexagonal (portas e adaptadores) dentro de cada módulo, orientado a metadados e com pontos de extensão baseados em registros.

Decisões de base

Decisão	Escolha	ADR
Estilo	Monolito modular	ADR-0001
Linguagem	TypeScript em cliente, servidor e pacotes compartilhados	ADR-0002
Banco	PostgreSQL, campos customizados em JSONB	ADR-0003
Comunicação interna assíncrona	Eventos de domínio + Transactional Outbox	ADR-0004
Fila de jobs	pg-boss sobre o próprio Postgres	ADR-0005
Instanciação de template	Cópia versionada (não referência viva)	ADR-0006
Repetição espaçada	FSRS	ADR-0007
Recorrência de eventos	RRULE (RFC 5545) + exceções	ADR-0008

Estrutura do repositório

```
estudos/
├── apps/
│   ├── web/                # frontend PWA
│   ├── api/                # servidor HTTP
│   └── worker/             # processamento assíncrono
├── modules/
│   ├── identity/
│   ├── workspaces/         # inclui templates e instanciação
│   ├── calendar/
│   ├── study/
│   ├── practice/
│   ├── notes/
│   ├── rules/
│   └── analytics/
├── packages/
│   ├── core/               # tipos e primitivas do núcleo (TS puro)
│   ├── registries/         # contratos + implementações dos catálogos
│   ├── template-schema/    # meta-schema de templates e validação
│   ├── db/                 # schema e migrações (Drizzle)
│   └── ui/                 # componentes compartilhados
├── templates/
│   ├── residencia-medica/
│   ├── concurso-publico/
│   └── graduacao/
├── tools/
│   └── generators/         # modelos de scaffolding (Plop)
└── docs/
    ├── ARQUITETURA.md
    ├── EXTENDING.md        # receitas de extensão
    └── adr/
```

---

4. Módulos e fronteiras

4.1 Responsabilidades

Módulo	Responsável por	Não é responsável por
`identity`	Contas, autenticação, sessão, papéis no workspace	Preferências de estudo
`workspaces`	Workspaces, templates, versões, instanciação, metadados (tipos, campos, widgets)	Os dados que os tipos descrevem
`calendar`	Eventos, recorrência, bloqueios de período, exportação `.ics`	Decidir quando estudar
`study`	Disciplinas, tópicos, itens de estudo (revisões, leituras), geradores de cronograma	Questões e tentativas
`practice`	Questões, tentativas, cartões e agendamento FSRS	Estatísticas agregadas
`notes`	Anotações em Markdown e vínculos com outras entidades	Conteúdo de questões
`rules`	Avaliação e execução de regras do usuário	Lógica de negócio dos outros módulos
`analytics`	Projeções e agregações para widgets e relatórios	Escrita em dados de outros módulos

4.2 Estrutura interna de cada módulo

```
modules/<nome>/
├── domain/        # entidades, objetos de valor, regras de negócio, eventos de domínio
├── application/   # casos de uso, portas (interfaces) de entrada e saída
├── adapters/      # repositórios Postgres, controladores HTTP, handlers de job
└── index.ts       # interface pública do módulo — a única coisa importável de fora
```

4.3 Regras de dependência

- **R-01** `domain/` **NÃO DEVE** importar nada de `application/`, `adapters/`, `packages/db` ou bibliotecas de infraestrutura (HTTP, ORM, fila).
- **R-02** `application/` **NÃO DEVE** importar de `adapters/`. Depende apenas de portas (interfaces) que os adaptadores implementam.
- **R-03** Um módulo **NÃO DEVE** importar arquivos internos de outro módulo. Apenas o `index.ts` público.

- **R-04** Um módulo **NÃO DEVE** ler ou escrever tabelas pertencentes a outro módulo. Cada tabela tem exatamente um módulo dono.
- **R-05** O grafo de dependências entre módulos **DEVE** ser acíclico e seguir a matriz abaixo.
- **R-06** `packages/core` e `packages/registries` **NÃO DEVEM** depender de nenhum módulo.

Matriz de dependências permitidas (linha pode depender de coluna):

	identity	workspaces	calendar	study	practice	notes	rules	analytics
identity	—							
workspaces	✓	—						
calendar	✓	✓	—					
study	✓	✓	✓	—				
practice	✓	✓		✓	—			
notes	✓	✓				—		
rules	✓	✓	✓	✓	✓	✓	—	
analytics	✓	✓						—

Observações: `analytics` consome apenas **eventos de domínio** dos demais módulos, nunca suas interfaces de escrita. Nenhum módulo depende de `rules` ou de `analytics`; eles reagem por eventos. `notes` referencia outras entidades apenas por `(tipo, id)`, sem chamar seus módulos.

4.4 Comunicação entre módulos

- **R-07** Chamadas síncronas entre módulos **DEVEM** passar pela interface pública e acontecer dentro do mesmo processo.
- **R-08** Efeitos colaterais em outro módulo (ex.: uma tentativa errada gerar revisão) **DEVEM** acontecer por evento de domínio, não por chamada direta do módulo de origem.
- **R-09** Eventos de domínio **DEVEM** ser imutáveis, versionados (`AttemptRegistered.v1`) e carregar `workspace_id`, `occurred_at`, `causation_id` e `correlation_id`.

---

5. Núcleo de domínio

O núcleo é o conjunto fixo de entidades. **Todo** recurso do produto precisa ser expresso com elas.

Entidade	Descrição	Customizável por template
`Workspace`	Unidade de organização ("Residência SES-DF 2027"). Dono dos demais dados	Nome, configurações, módulos ativos
`EventType`	Tipo de compromisso (Aula, Plantão, Simulado, Saída de campo)	Totalmente
`Event`	Ocorrência no calendário, com início, fim, recorrência e campos customizados	Campos, via `EventType`
`Subject` / `Topic`	Árvore de conteúdo (disciplina → tópico)	Estrutura e campos
`ItemType`	Tipo de item de estudo (Revisão, Leitura, Lista de exercícios)	Totalmente
`StudyItem`	Unidade de trabalho com status e data prevista	Campos, via `ItemType`
`Question` / `Attempt`	Questão e cada resposta dada a ela	Campos da questão
`Card` / `ReviewLog`	Cartão de repetição espaçada e seu histórico	Não
`Note`	Anotação em Markdown com vínculos	Não
`Goal`	Meta mensurável (horas, questões, itens concluídos)	Métrica e período

- **R-10** Uma nova entidade no núcleo **SÓ PODE** ser criada se for necessária a **todos** os perfis de estudante (ver régua em §15.2) e exige ADR.
- **R-11** O núcleo **NÃO DEVE** conter nomes, enums ou condicionais de áreas específicas (`'plantao'`, `'medicina'`, `'edital'`). Esses termos só podem existir em `templates/`.

6. Metadados e templates

6.1 O que é um template

Um template é um pacote **declarativo** e **versionado** (SemVer) contendo:

- tipos de evento e de item, com seus campos;
- taxonomia inicial (disciplinas e tópicos);
- configuração de geradores de cronograma;
- regras sugeridas (desativadas por padrão);
- composição do painel (widgets);

- rótulos e ícones.

Exemplo resumido:

```json
{
  "id": "residencia-medica",
  "version": "1.0.0",
  "name": "Residência Médica",
  "eventTypes": [
    {
      "key": "plantao",
      "label": "Plantão",
      "icon": "hospital",
      "defaultDurationMin": 720,
      "fields": [
        { "key": "local", "type": "select", "options": ["HRC", "HRAN", "HBDF"]
        { "key": "turno", "type": "select", "options": ["diurno", "noturno", "2
      ]
    }
  ],
  "generators": [{ "kind": "linear", "params": { "itemsPerWeek": 4 } }],
  "dashboard": ["progress", "events-of-type:plantao", "upcoming:aula"]
}
```

6.2 Regras de templates

- **R-12** Todo template **DEVE** validar contra o meta-schema em `packages/template-schema` antes de ser publicado ou instanciado.
- **R-13** Um template **NÃO DEVE** conter código executável, expressões avaliáveis, HTML ou CSS. Apenas dados que referenciam entradas de registros.
- **R-14** Toda referência em um template (`"type": "select"`, `"kind": "linear"`) **DEVE** apontar para uma entrada existente no registro correspondente. Referência desconhecida invalida o template.
- **R-15** Instanciar um template **DEVE** copiar seus metadados para o workspace e gravar `template_id` + `template_version` de origem. O workspace nunca lê o template ao vivo.
- **R-16** Uma nova versão de template **NÃO DEVE** ser aplicada automaticamente. A atualização é oferecida ao usuário como diff, aceito campo a campo.
- **R-17** Remover um tipo ou campo que já possui dados **DEVE** arquivá-lo (soft delete). Os valores existentes são preservados e continuam exportáveis.
- **R-18** Templates compartilhados entre usuários **NÃO DEVEM** levar dados do workspace de origem (eventos, tentativas, notas). Apenas metadados e taxonomia.
- **R-19** Mudanças incompatíveis no meta-schema **DEVEM** vir com migração automática dos workspaces existentes.

---

7. Registros (pontos de extensão)

Registros são catálogos fechados de implementações. São o **único** lugar onde o comportamento de algo customizável é definido.

Registro	Exemplos de entradas iniciais
`FieldTypeRegistry`	`text`, `long-text`, `number`, `select`, `multi-select`, `date`, `boolean`, `duration`, `checklist`, `attachment`, `ref-topic`, `ref-event`
`GeneratorRegistry`	`linear`, `weighted`, `cycle`, `until-exam`, `by-performance`
`TriggerRegistry`	`event.created`, `event.finished`, `item.completed`, `attempt.registered`, `date.approaching`
`ConditionRegistry`	`field-equals`, `field-in`, `rate-below`, `count-above`, `type-is`
`ActionRegistry`	`create-item`, `block-period`, `notify`, `reschedule`, `run-generator`
`WidgetRegistry`	`progress`, `hours-by-subject`, `events-of-type`, `countdown`, `accuracy-by-topic`

7.1 Contrato de um tipo de campo

ts

```ts
export interface FieldType<TConfig, TValue> {
  readonly key: string;                          // "select"
  readonly configSchema: JSONSchema;             // valida a configuração (ex.:
  valueSchema(config: TConfig): JSONSchema;      // valida o valor salvo
  defaultValue(config: TConfig): TValue | null;
  isAnalyzable: boolean;                         // pode virar dimensão de anál
  toIndexExpression?(fieldPath: string): string; // SQL para índice, se analisá
}

// Parte de UI, registrada apenas em apps/web
export interface FieldTypeView<TConfig, TValue> {
  readonly key: string;
  Editor: Component<{ config: TConfig; value: TValue; onChange(v: TValue): void
  Display: Component<{ config: TConfig; value: TValue }>;
  Filter?: Component<{ config: TConfig; onChange(f: FilterExpr): void }>;
}
```

7.2 Regras de registros

- **R-20** Toda entrada de registro **DEVE** ter `key` única, estável e imutável após o primeiro release. Renomear exige nova chave e migração.
- **R-21** A parte de domínio de um registro **DEVE** ficar em `packages/registries` e ser usada igualmente pelo cliente e pelo servidor. A validação de um valor **NÃO DEVE** existir em duas implementações.
- **R-22** Toda entrada de registro **DEVE** ter testes cobrindo validação de configuração, validação de valor e casos-limite.
- **R-23** Adicionar uma entrada de registro **NÃO DEVERIA** exigir alteração em nenhum outro arquivo além do próprio arquivo e da linha de registro.
- **R-24** Geradores e ações **DEVEM** ser determinísticos dado o mesmo estado e os mesmos parâmetros, para permitir pré-visualização e testes.

---

8. Camadas de customização e limites

8.1 As três camadas

Camada	O usuário pode	Disponível em
1 — Configuração	Escolher template, renomear rótulos, trocar cores e ícones, ocultar módulos, editar a taxonomia	Fase 0 (MVP)
2 — Tipos e campos	Criar e editar tipos de evento e de item, com campos do catálogo; marcar campos como analisáveis; configurar widgets	Fase 3
3 — Geradores e regras	Configurar parâmetros de geradores; ativar regras sugeridas; criar regras com gatilhos, condições e ações do catálogo	Geradores: Fase 0 · Regras: Fase 4

8.2 Limites quantitativos (valores iniciais)

Os valores abaixo ficam em configuração e podem ser revistos com dados reais de uso, por ADR.

Recurso	Limite por workspace
Tipos de evento	50
Tipos de item	30
Campos por tipo	30
Campos analisáveis por tipo	5
Opções em `select` / `multi-select`	100
Itens em `checklist`	50
Texto curto	2.000 caracteres
Nota (Markdown)	100.000 caracteres
Anexo	10 MB por arquivo · 500 MB por workspace
Regras ativas	50
Condições por regra	5
Ações por regra	5
Profundidade de encadeamento de regras	3
Widgets no painel	12
Horizonte de um gerador	104 semanas
Itens/eventos criados por execução de gerador	5.000

- **R-25** Os limites **DEVEM** ser verificados no servidor. Verificações no cliente são apenas para experiência de uso.
- **R-26** Ao atingir um limite, o sistema **DEVE** recusar a operação com mensagem clara. **NÃO DEVE** truncar ou descartar dados silenciosamente.

8.3 Limites qualitativos

O que a customização **não** permite, em nenhuma camada:

- **L-01 Entidades novas.** Usuários não criam novos tipos de entidade com relações próprias. Tudo é evento, item, tópico, questão ou nota.
- **L-02 Código de usuário.** Sem scripts, fórmulas livres, expressões avaliadas ou plugins de terceiros. Campos calculados se limitam a agregações prontas (soma, média, contagem, taxa).

- **L-03 Tipos de campo novos.** Usuários combinam tipos do catálogo; não definem tipos novos.
- **L-04 Algoritmos novos.** Geradores, condições e ações novos são trabalho da equipe, entregues como entradas de registro.
- **L-05 Layout livre.** Painéis são compostos por widgets do catálogo, em grade responsiva. Sem HTML/CSS de usuário.
- **L-06 Análise sobre qualquer campo.** Apenas campos marcados como analisáveis entram em gráficos e agregações. Os demais servem para exibição e filtro simples.
- **L-07 Dados sensíveis de terceiros.** Nenhum template ou tipo pode ser desenhado para registrar dados de pacientes ou outras pessoas identificáveis (ver §12).

---

9. Motor de regras

9.1 Formato

```json
{
  "id": "r_01",
  "enabled": true,
  "when": { "trigger": "event.created" },
  "if": [
    { "condition": "type-is", "params": { "eventType": "plantao" } },
    { "condition": "field-equals", "params": { "field": "turno", "value": "notu
  ],
  "then": [
    { "action": "block-period", "params": { "offset": "PT0H", "anchor": "end",
  ]
}
```

Condições em `if` são combinadas por **E**. Para **OU**, cria-se outra regra.

9.2 Execução

1. O caso de uso grava a mudança e o evento de domínio na tabela `outbox`, **na mesma transação**.
2. O `worker` lê a outbox, encontra regras do workspace cujo gatilho corresponde e avalia as condições.
3. As ações são executadas pelas interfaces públicas dos módulos donos, propagando `causation_id` e `depth + 1`.
4. Cada execução é registrada em `rule_executions` (regra, evento de origem, ações, resultado).

9.3 Regras do motor

- **R-27** O processamento de eventos **DEVE** usar o padrão Transactional Outbox. Nenhuma regra roda sobre uma mudança que não foi persistida.

- **R-28** Handlers **DEVEM** ser idempotentes. Processar o mesmo evento duas vezes não pode duplicar efeitos.
- **R-29** Uma execução com `depth` acima do limite (3) **DEVE** ser interrompida e registrada como bloqueada.
- **R-30** Uma regra **NÃO DEVE** disparar a si mesma, direta ou indiretamente, dentro da mesma cadeia de `correlation_id`.
- **R-31** Toda entidade criada ou alterada por regra **DEVE** registrar a regra de origem, e a interface **DEVE** exibir essa informação ("criado pela regra *Descanso pós-plantão*").
- **R-32** Ações de regra **NÃO DEVEM** excluir dados. Apenas criar, reagendar, bloquear e notificar.
- **R-33** Uma regra que falhar 5 vezes seguidas **DEVE** ser desativada automaticamente, com aviso ao usuário.

---

10. Dados e persistência

- **R-34** Toda tabela de dados de usuário **DEVE** ter `id` (UUIDv7), `workspace_id`, `created_at`, `updated_at` e `deleted_at` (soft delete).
- **R-35** Toda consulta a dados de usuário **DEVE** filtrar por `workspace_id`. Repositórios recebem o workspace como parâmetro obrigatório; não há método "buscar todos".
- **R-36** Campos customizados **DEVEM** ficar em uma coluna `custom_fields jsonb`, validada no servidor contra o JSON Schema do tipo (Ajv) antes de qualquer escrita.
- **R-37** Campos marcados como analisáveis **DEVEM** ter índice de expressão criado pelo sistema. Índices sobre JSONB não analisável **NÃO DEVEM** ser criados ad hoc.
- **R-38** O modelo EAV (tabela genérica chave/valor) **NÃO DEVE** ser usado.
- **R-39** Migrações **DEVEM** ser versionadas em `packages/db`, reversíveis quando possível e aplicadas pelo pipeline, nunca manualmente.
- **R-40** Datas e horas **DEVEM** ser armazenadas em UTC (`timestamptz`), com o fuso do workspace salvo à parte (padrão `America/Sao_Paulo`). Recorrências usam RRULE no fuso do workspace.
- **R-41** O workspace **DEVE** ser exportável por completo em JSON versionado, incluindo metadados, e reimportável. O importador **DEVE** aceitar o formato `cronograma.json` do protótipo original.

---

11. Frontend, offline e sincronização

- **R-42** Formulários, filtros e exibições de tipos customizados **DEVEM** ser gerados a partir dos metadados e do `FieldTypeView` registrado. **NÃO DEVE** haver telas específicas por template.
- **R-43** O frontend **NÃO DEVE** conter condicionais por template ou área (`if (template === ...)`).
- **R-44** A aplicação **DEVE** ser uma PWA instalável, com acessibilidade mínima WCAG 2.1 AA: alvos de toque de 44 px, contraste adequado, navegação por teclado e rótulos ARIA (o protótipo original já segue esse padrão e serve de referência).
- **R-45** IDs de novas entidades **DEVEM** ser gerados no cliente (UUIDv7), para permitir criação offline.

- **R-46** No estágio inicial, o suporte offline **DEVE** se limitar a: leitura a partir de cache persistido (IndexedDB) e fila local de mutações reenviada ao reconectar.
- **R-47** Conflitos de escrita **DEVEM** ser resolvidos por *last-write-wins* por campo, usando `updated_at`. Sincronização local-first completa (CRDT) está fora de escopo até um ADR específico.

---

12. Segurança e privacidade

- **R-48** Autorização **DEVE** ser verificada por caso de uso, com base no papel do usuário no workspace (`owner`, `editor`, `viewer`). Se o banco suportar, Row Level Security por `workspace_id` é camada adicional, não substituta.
- **R-49** Texto livre de usuário (notas, campos) **DEVE** ser sanitizado na renderização. Markdown é renderizado sem HTML bruto.
- **R-50** Anexos **DEVEM** ser servidos por URLs assinadas e temporárias, com verificação de tipo e tamanho no upload.
- **R-51** Templates oficiais **NÃO DEVEM** sugerir campos para dados de saúde, identificação ou contato de terceiros. A interface de criação de campos **DEVE** exibir aviso sobre dados pessoais de terceiros.
- **R-52** O usuário **DEVE** poder excluir sua conta e todos os seus workspaces, com remoção definitiva após o período de retenção documentado (LGPD).
- **R-53** Logs **NÃO DEVEM** conter conteúdo de notas, campos customizados ou anexos. Apenas identificadores e metadados técnicos.

---

13. Extensibilidade

A arquitetura foi desenhada para que **ampliar seja, na maioria dos casos, adicionar, e não modificar**. Ela é deliberadamente barata em algumas direções e deliberadamente mais cara em outras.

13.1 Tipos de extensão

Tipo de extensão	Onde entra	Esforço	Toca código existente?	Exemplo
Template novo	`templates/`	Baixo	Não (só dados)	Vestibular, Residência em Odontologia
Tipo de campo	`FieldTypeRegistry`	Baixo	Só a linha de registro	Avaliação em estrelas
Widget	`WidgetRegistry` + projeção em `analytics`	Baixo	Só a linha de registro	Sequência de dias estudados
Gerador, gatilho, condição ou ação	Registro correspondente	Baixo a médio	Só a linha de registro	Gerador "ciclo de estudos"
Integração externa	Novo adaptador no módulo dono	Baixo a médio	Não (o domínio não muda)	Google Calendar, notificações push
Módulo que reage a fatos existentes	Módulo novo consumindo eventos	Médio	Não	Gamificação, conquistas
Entidade nova no núcleo	Novo módulo ou módulo existente + ADR	Médio a alto	Sim, em vários pontos	Sessões de estudo (cronômetro)
Mudança no formato de template	`template-schema` + migração	Alto	Sim	Metas sugeridas por template
Colaboração em tempo real	ADR de sincronização	Alto	Sim	Grupos de estudo editando juntos

13.2 Pontos de atenção conhecidos

- **Indireção.** O comportamento de um conceito de área (ex.: plantão) fica distribuído entre template, registros e regras. Não existe um `PlantaoService`. O `EXTENDING.md` e o scaffolding existem para compensar isso.
- **Entidades novas atravessam muitas camadas.** Por isso seguem uma checklist obrigatória (§13.3).
- **Evolução do meta-schema** é a mudança mais cara do sistema, porque exige migrar workspaces existentes (R-19). Deveria ser rara e agrupada.

13.3 Checklist para entidade nova no núcleo

Uma entidade nova só está concluída quando todos os itens abaixo estiverem feitos no mesmo PR (ou em PRs ligados ao mesmo ADR):

- [ ] ADR aprovado justificando a entrada no núcleo (R-10)
- [ ] Módulo dono definido e matriz de dependências atualizada, se necessário
- [ ] Tabela com `id`, `workspace_id`, `created_at`, `updated_at`, `deleted_at` (R-34) e migração (R-39)
- [ ] Repositório com `workspace_id` obrigatório (R-35)
- [ ] Eventos de domínio versionados (R-09)
- [ ] Autorização por papel nos casos de uso (R-48)
- [ ] Inclusão na exportação e na importação do workspace (R-41)
- [ ] Suporte no cache offline e na fila de mutações (R-46)
- [ ] Testes de isolamento por workspace
- [ ] Gatilhos e widgets do catálogo atualizados, se a entidade deve ser usável por regras ou painéis
- [ ] Receita ou seção correspondente no `EXTENDING.md`

13.4 Regras de extensibilidade

- **R-54** Toda extensão de um tipo listado em §13.1 **DEVE** seguir a receita correspondente em `docs/EXTENDING.md`. Se a receita não existir, ela **DEVE** ser escrita no mesmo PR da primeira extensão daquele tipo.
- **R-55** Entidades novas no núcleo **DEVEM** cumprir a checklist da §13.3.
- **R-56** Módulos novos **DEVERIAM** partir da estrutura do módulo de referência (`modules/notes`), mantido simples e comentado para esse fim.

---

14. Ferramentas de desenvolvimento

14.1 Scaffolding

Scaffolding é a geração automática da estrutura inicial de arquivos a partir de modelos. O projeto usa Plop (em `tools/generators/`) para que o caminho que segue a arquitetura seja também o caminho mais fácil.

```bash
pnpm gen field-type     # domínio + UI + testes + linhas de registro
pnpm gen widget
pnpm gen generator
pnpm gen action
pnpm gen template       # pasta em templates/ com template mínimo válido
pnpm gen module         # estrutura domain/application/adapters/index.ts
pnpm gen core-entity    # todos os pontos da checklist §13.3, com TODOs
```

Os arquivos gerados já implementam a interface correspondente pela metade e trazem os casos de teste obrigatórios (R-22) esperando implementação.

14.2 Regras

- **R-57** Um gerador de scaffolding **DEVERIA** ser criado apenas depois de existirem de dois a três exemplos feitos à mão do mesmo tipo, para não congelar um padrão ainda imaturo.
- **R-58** Quando um contrato de registro ou a estrutura de módulo mudar, o modelo de scaffolding correspondente **DEVE** ser atualizado no mesmo PR.
- **R-59** O código gerado por scaffolding **DEVE** passar no lint e nas verificações de fronteira sem edição manual.

---

15. Processo de mudança

15.1 ADRs

Decisões que afetam este documento são registradas em `docs/adr/NNNN-titulo.md`, com contexto, decisão, alternativas consideradas e consequências. São obrigatórias para: nova entidade no núcleo, novo módulo, mudança na matriz de dependências, alteração de limites, mudança de tecnologia de base, e exceções a regras **DEVE**.

15.2 Onde um requisito novo entra

Pergunta	Se sim	Exemplo
Todo perfil de estudante precisa?	**Núcleo** (código + ADR)	Repetição espaçada
Uma área inteira precisa?	**Template** (dados)	Plantão, simulado, saída de campo
Precisa de um comportamento que o catálogo não tem?	**Nova entrada de registro** (código)	Gerador "ciclo de estudos"
Só uma pessoa precisa?	**Customização do usuário** (campo, tag, nota)	Campo "bancada" no laboratório
Exige entidade nova ou código de usuário?	**Fora de escopo** (reavaliar por ADR)	Prontuário de pacientes

15.3 Critérios para extrair um módulo para serviço próprio

Só por ADR, e apenas se houver ao menos um destes sinais medidos: necessidade de escalar independentemente (ex.: worker saturado), requisito de isolamento de falha comprovado, ou equipe dedicada ao módulo. A extração natural prevista é `apps/worker`, que já roda em processo separado.

---

16. Verificação automática (fitness functions)

Regras sem verificação se perdem. O CI **DEVE** executar:

Verificação	Ferramenta	Regras cobertas
Fronteiras entre camadas e módulos	`dependency-cruiser` ou `eslint-plugin-boundaries`	R-01 a R-06
Termos de área fora de `templates/`	lint customizado (lista de termos proibidos no núcleo)	R-11, R-43
Validação de todos os templates	teste contra o meta-schema	R-12 a R-14
Contrato de templates	instanciar cada template e comparar com snapshot esperado	R-15, R-24
Paridade com o protótipo	o template `residencia-medica` com gerador `linear` gera as mesmas semanas que o HTML original	R-41
Propriedades do motor de regras	`fast-check`: combinações aleatórias nunca excedem a profundidade nem entram em laço	R-29, R-30
Idempotência de handlers	reprocessar eventos e comparar estado	R-28
Isolamento por workspace	testes que tentam ler dados de outro workspace	R-35, R-48
Acessibilidade	`axe-core` nas telas principais	R-44
Scaffolding válido	gerar uma entrada de cada tipo no CI e rodar lint e testes	R-58, R-59

17. Implementação por fases

A arquitetura completa não precisa existir no primeiro dia. Cada fase define o escopo de produto e quais regras passam a ser verificadas em CI. Regras de fases futuras valem como direção desde já, mas só bloqueiam o build quando sua fase começar.

Fase	Escopo	Regras verificadas em CI a partir desta fase
0 — MVP	Núcleo mínimo (`workspaces`, `calendar`, `study`), template `residencia-medica` portado do protótipo, gerador `linear`, customização da camada 1, PWA, importação do `cronograma.json`	R-01 a R-03, R-11 a R-15, R-34 a R-36, R-41, R-44, paridade com o protótipo
1 — Contas e segundo template	`identity`, sincronização entre dispositivos, template `concurso-publico`, gerador `weighted`	R-04 a R-06 (matriz completa), R-45 a R-48
2 — Prática	`practice` (questões, tentativas, FSRS), `analytics`, widgets de desempenho	R-07 a R-09, R-27, R-28, R-37
3 — Customização avançada	Camada 2 (tipos e campos do usuário), templates compartilháveis, exportação `.ics`, scaffolding	R-16 a R-26, R-54 a R-59
4 — Automação	Motor de regras (camada 3 completa)	R-29 a R-33

O template `concurso-publico` na fase 1 é intencional: o segundo template é o que revela o que ainda está específico demais no núcleo.

---

18. Fora de escopo

Nesta fase, explicitamente **não** faremos:

- microsserviços ou múltiplos bancos;
- scripts, fórmulas livres ou plugins de usuário;
- sincronização local-first com CRDT;
- colaboração em tempo real na mesma nota;
- marketplace público de templates com moderação;
- banco de questões com conteúdo de plataformas comerciais;
- aplicativos nativos (o caminho previsto é Capacitor sobre a PWA).

---

19. Glossário

Termo	Significado
Workspace	Espaço de estudo de um objetivo, com seu template, metadados e dados
Template	Pacote declarativo e versionado de metadados para uma área de estudo
Metadados	Definições que descrevem os dados: tipos, campos, regras, widgets
Registro	Catálogo fechado de implementações (tipos de campo, geradores, ações...)
Gerador	Estratégia que cria itens ou eventos a partir de parâmetros (ex.: 4 tópicos por semana)
Regra	Automação *quando → se → então* montada com peças dos registros
Campo analisável	Campo customizado indexado e disponível como dimensão em widgets e relatórios
Outbox	Tabela onde eventos de domínio são gravados na mesma transação da mudança
Causation / correlation id	Identificadores que ligam um efeito ao evento que o causou e à cadeia inteira
Scaffolding	Geração automática da estrutura inicial de arquivos a partir de modelos
Módulo de referência	Módulo mantido simples e comentado para servir de modelo a módulos novos
ADR	*Architecture Decision Record*: registro curto de uma decisão arquitetural, com contexto e alternativas